



LEARN
WITH
SARVESH

COURSES • CLARITY • CONFIDENCE

PYTHON • INTERVIEW PREP • 2026

TOP 10 PYTHON INTERVIEW QUESTIONS

Beginner-friendly answers, explained simply.

01QUESTION
01 of 10

What's the difference between a list and a tuple?

ANSWER

A list is mutable — you can add, remove or change its items after creating it, and it uses square brackets. A tuple is immutable — once made it can't be changed, and it uses parentheses. Tuples are a bit faster and can even be used as dictionary keys.



```
nums = [1, 2, 3]    # list -> can change  
point = (4, 5)     # tuple -> fixed
```

02QUESTION
02 of 10

What are `*args` and `**kwargs`?

ANSWER

They let a function accept any number of arguments. `*args` collects extra positional arguments into a tuple, while `**kwargs` collects extra keyword arguments into a dictionary. Great when you don't know how many inputs you'll get.



```
def demo(*args, **kwargs):  
    print(args)      # (1, 2)  
    print(kwargs)    # {'name': 'Sam'}
```


**03**QUESTION
03 of 10

What's the difference between == and is?

ANSWER

== compares values and asks 'are these equal?'. is compares identity and asks 'are these the exact same object in memory?'. Two things can be equal in value yet still be separate objects.



```
a = [1, 2]; b = [1, 2]
a == b    # True   (same values)
a is b    # False  (different objects)
```

**04**QUESTION
04 of 10

What is a list comprehension?

ANSWER

A short, readable way to build a list in a single line instead of writing a full for-loop. It's quicker to write, easier to read, and often runs a little faster too.



```
squares = [n * n for n in range(5)]  
# [0, 1, 4, 9, 16]
```

05QUESTION
05 of 10

Mutable vs immutable objects?

ANSWER

Mutable objects can be changed after they're created — like lists, dictionaries and sets. Immutable objects can't be changed — like integers, strings and tuples. Knowing which is which helps you avoid sneaky bugs.



```
s = 'hi'; s[0] = 'H'    # error - can't change  
lst = [1, 2]; lst[0] = 9 # ok - can change
```

06QUESTION
06 of 10

What is a lambda function?

ANSWER

A lambda is a small, anonymous function you can write in one line. It's handy for quick, throwaway logic — often passed straight into helpers like map, filter or sorted.



```
square = lambda x: x * x  
square(6)    # 36
```

07QUESTION
07 of 10

Difference between `append()` and `extend()`?

ANSWER

`append()` adds whatever you pass as one single item at the end of the list. `extend()` takes an iterable and adds each of its elements individually. Same-looking calls, very different results.



```
a = [1, 2]; a.append([3, 4]) # [1, 2, [3, 4]]  
b = [1, 2]; b.extend([3, 4]) # [1, 2, 3, 4]
```


08QUESTION
08 of 10

What does `if __name__ == '__main__':` do?

ANSWER

It lets a block of code run only when the file is executed directly, and skips it when the file is imported elsewhere. It's the standard way to separate reusable code from a script's entry point.



```
if __name__ == '__main__':  
    main()    # runs only when run directly
```

09QUESTION
09 of 10

How does a Python dictionary work?

ANSWER

A dictionary stores data as key–value pairs. Keys must be unique and immutable, and Python uses a hash table underneath, so looking up a value by its key is extremely fast — even in huge dictionaries.



```
user = {'name': 'Sam', 'age': 25}  
user['name']    # 'Sam'
```

10QUESTION
10 of 10

What is a decorator?

ANSWER

A decorator is a function that wraps another function to add extra behaviour — like logging or timing — without touching the original code. You apply it neatly with the @ symbol above a function.

```

@log_time
def run():
    ... # @log_time adds timing around this
```



LEARN
WITH
SARVESH

COURSES CLARITY CONFIDENCE

YOU JUST LEVELED UP YOUR PYTHON PREP

Save this post for your next interview and
pass it to a friend who's preparing too.



for more insights on AI, LLMs & GenAI.

🌐 learnwithsarvesh.com